**EX. NO.: 1(a)** **STUDY OF NETWORKING COMMANDS**

**Date:** _____

**AIM:**

To study and understand the various basic networking commands used to view, analyze, configure, and troubleshoot network connections in a computer system.

**INTRODUCTION:**

Networking commands are essential tools for **network administrators and engineers** to diagnose, monitor, and manage network configurations. These commands help in identifying connection issues, viewing IP details, monitoring data packets, and managing routing information. They are used in both **Windows** and **Linux** environments for testing connectivity, resolving DNS issues, and ensuring smooth communication between devices in a network.

**COMMANDS AND DESCRIPTION:**

**1. arp –a**

Displays the **Address Resolution Protocol (ARP)** table, showing IP and MAC addresses of devices currently communicating with your computer.
**Use:** Helps verify physical address mappings and detect unauthorized devices in a network.

**2. hostname**

Displays the **name of the current computer**.
**Use:** Useful when identifying a machine in a network, especially in systems with multiple hosts.

**3. ipconfig**

Shows the **basic TCP/IP configuration** such as IP address, subnet mask, and default gateway.
**Use:** Commonly used to check the network configuration of a system.

**4. ipconfig /all**

Displays detailed TCP/IP configuration, including **DNS, DHCP, adapter type**, and **MAC address**.
**Use:** Used for detailed network analysis and verifying automatic IP assignments.

**5. ipconfig /renew**

Requests a new IP address from the **DHCP (Dynamic Host Configuration Protocol)** server.
**Use:** Helps when a system has lost connection or needs a new IP due to network changes.

**6. ipconfig /release**

Releases the current IP address assigned by the DHCP server.
**Use:** Used before obtaining a new IP address or when disconnecting from a network.

**7. ipconfig /flushdns**

Clears the **DNS resolver cache**, removing outdated or incorrect DNS entries.
**Use:** Helpful when websites fail to load due to old or incorrect DNS records.

**8. nbtstat –a**

Displays the **NetBIOS over TCP/IP** connections and helps resolve NetBIOS name issues.
**Use:** Used to troubleshoot name resolution and connectivity problems in local networks.

**9. netdiag**

Performs various **network diagnostic tests** to identify and resolve problems.
**Use:** Useful for comprehensive testing of network adapters and configurations.

**10. netstat**

Displays statistics about **active TCP/UDP connections**, listening ports, and network interface data.
**Use:** Helps monitor open connections and detect suspicious or unauthorized traffic.

**11. nslookup**

Used to **query DNS servers** and obtain domain name or IP address mapping.
**Use:** Essential for diagnosing DNS problems and verifying domain records.

**12. pathping**

Combines the functions of **ping** and **tracert** to show the network path and measure packet loss at each hop.
**Use:** Detects which router or link causes delays or packet drops.

**13. ping**

Sends **ICMP Echo Request** packets to a destination host to test connectivity and response time.
**Use:** Basic and most common command to check if a network or host is reachable.

**14. route**

Displays or modifies entries in the **local routing table**.
**Use:** Helps configure static routes and understand packet forwarding paths.

**15. tracert**

Traces the route packets take to reach a destination host by showing all intermediate routers.
**Use:** Used to identify network path problems or slow network hops.

**16. tcpdump Commands**

Used in **Linux/Unix systems** for packet capturing and network traffic analysis.

- **$ tcpdump -i any** – Captures packets from all available interfaces.
  *Use:* Helpful when multiple network cards are active.
- **$ tcpdump -i eth0** – Captures packets from a specific interface (e.g., Ethernet 0).
  *Use:* Focuses capture on a single connection type.
- **$ tcpdump -r packets_file** – Reads packets from a saved file for offline analysis.
  *Use:* Used in network forensics and performance review.
- **$ tcpdump net 192.168.1.0/24** – Captures packets from all devices in a specific network.
  *Use:* Monitors local network traffic for security or debugging.
- **$ tcpdump src 192.168.1.100** – Captures packets originating from a specific source IP.
- **$ tcpdump dst 192.168.1.100** – Captures packets destined to a specific IP.
- **$ tcpdump ssh** – Captures packets related to the **SSH protocol**.
  *Use:* Monitors secure shell connections.
- **$ tcpdump port 22** – Captures packets on a specific port (SSH).
- **$ tcpdump portrange 22-125** – Captures packets across a range of port numbers.

**APPLICATIONS OF NETWORK COMMANDS:**

1. **Network Configuration** – Setting IP addresses, gateways, and DNS servers.
2. **Troubleshooting** – Identifying and resolving connectivity issues.
3. **Performance Monitoring** – Checking latency, bandwidth usage, and active sessions.
4. **Security Analysis** – Monitoring suspicious connections and packet flow.
5. **Server Management** – Ensuring proper routing, DNS resolution, and uptime.

**RESULT:**

Thus, the basic networking commands were studied in detail, and their functions, uses, and importance in network configuration and troubleshooting were successfully understood.

**EX. NO.: 1(b)** **SIMULATION OF PING COMMAND**

**Date:**

**AIM:**

To simulate the ping command using Java language.

**OUTCOME:**

Students gain knowledge in understanding HTTP client/server communication and implementing it using Java.

**ALGORITHM:**

1. Start the program.
2. Create an object for the class Process.
3. Get the IP address from the user using getInputStream().
4. Execute the ping command using Runtime.getRuntime().exec().
5. Stop the program.

**PROGRAM:**

```
import java.util.*;
public class PingDemo {
   public void ping(String host) {
      try {
         Process p = Runtime.getRuntime().exec("ping " + host);
         Scanner scan = new Scanner(p.getInputStream());
         while (scan.hasNextLine()) {
            System.out.println(scan.nextLine());
         }
```

```
        } catch (Exception ex) {
            System.out.println("Error " + ex);
        }
    }
    public static void main(String args[]) {
        PingDemo p = new PingDemo();
        p.ping(args[0]);
    }
}
```

**OUTPUT:**




**RESULT:**

Thus, the Java program to simulate the ping command was executed successfully.

**EX. NO.: 1(c)** **SIMULATION OF TRACEROUTE COMMAND**

**Date:**

**AIM:**

To simulate the traceroute command using Java language.

**OUTCOME:**

Students gain knowledge of traceroute command implementation using Java.

**ALGORITHM:**

1. Start the program.
2. Create an object for the class Process.
3. Get the IP address from the user using getInputStream().
4. Execute the tracert command using Runtime.getRuntime().exec().
5. Stop the program.

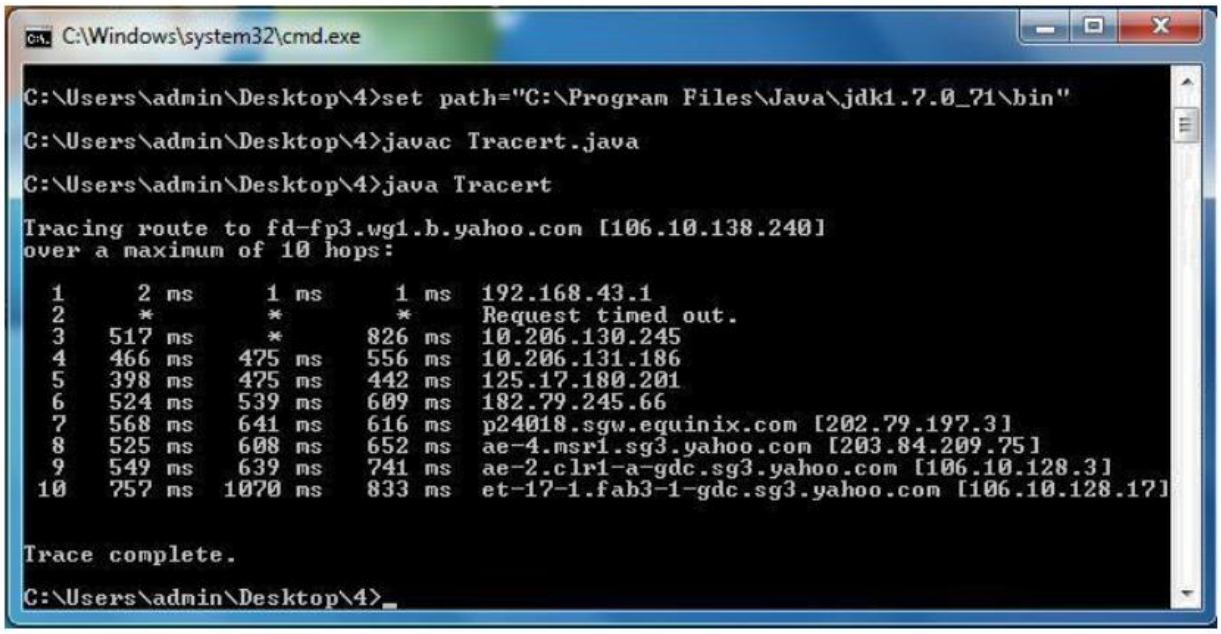
**PROGRAM:**

```
import java.util.*;
public class TraceDemo {
   public void trace(String host) {
      try {
         Process p = Runtime.getRuntime().exec("tracert " + host);
         Scanner scan = new Scanner(p.getInputStream());
         while (scan.hasNextLine()) {
            System.out.println(scan.nextLine());
         }
      } catch (Exception ex) {
```

```
            System.out.println("Error " + ex);
        }
    }

    public static void main(String args[]) {
        TraceDemo p = new TraceDemo();
        p.trace(args[0]);
    }
}
```

**OUTPUT:**




**RESULT:**

Thus, the Java program to simulate the traceroute command was executed successfully.

**EX. NO: 2** **CREATE SOCKET FOR HTTP FOR WEB PAGE UPLOAD AND DOWNLOAD**

**Date:**

**AIM:**

To create an HTTP socket for web page upload and download using Java programming.

**OUTCOME:**

Students gain knowledge in understanding subnetting and implementing it using Java.

**ALGORITHM:**

1. Start the program.
2. Read the input file name.
3. Create a connection with the HTTP server.
4. Log in to the HTTP server with username and password.
5. Set the upload file path on the HTTP server.
6. Upload the file.
7. Read the name of the file to be downloaded and the path where the file will be stored.
8. Download the file stream and store it in the specified path.
9. Stop the program.

**PROGRAM:**

**Client.java**

```
import javax.swing.*;
import java.net.*;
import java.awt.image.*;
import javax.imageio.*;
import java.io.*;
```

```
public class Client {
    public static void main(String args[]) throws Exception {
        Socket soc;
        BufferedImage img = null;
        soc = new Socket("localhost", 4000);
        System.out.println("Client is running.");

        try {
            System.out.println("Reading image from disk.");
            img = ImageIO.read(new File("digital_image_processing.jpg"));
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            ImageIO.write(img, "jpg", baos);
            baos.flush();
            byte[] bytes = baos.toByteArray();
            baos.close();

            System.out.println("Sending image to server.");
            OutputStream out = soc.getOutputStream();
            DataOutputStream dos = new DataOutputStream(out);
            dos.writeInt(bytes.length);
            dos.write(bytes, 0, bytes.length);
            System.out.println("Image sent to server.");

            dos.close();
            out.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
```

```
            soc.close();
        }
        soc.close();
    }
}
```

**Server.java**

```
import java.net.*;
import java.io.*;
import java.awt.image.*;
import javax.imageio.*;
import javax.swing.*;

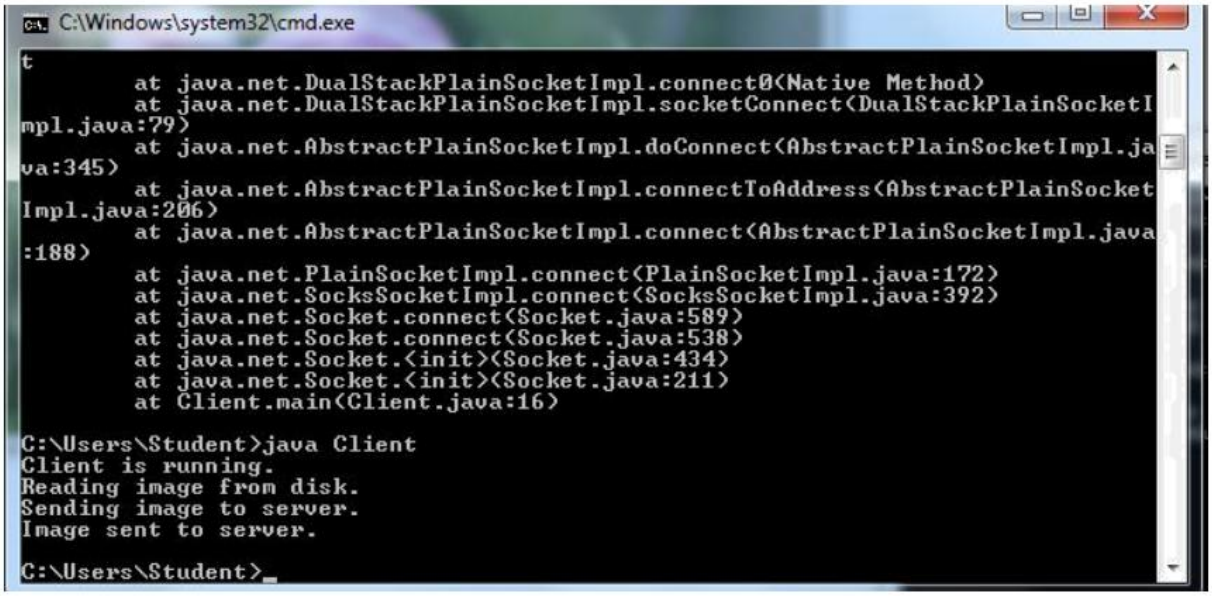
class Server {
    public static void main(String args[]) throws Exception {
        ServerSocket server = new ServerSocket(4000);
        System.out.println("Server Waiting for image");
        Socket socket = server.accept();
        System.out.println("Client connected.");

        InputStream in = socket.getInputStream();
        DataInputStream dis = new DataInputStream(in);
        int len = dis.readInt();
        System.out.println("Image Size: " + len/1024 + " KB");
        byte[] data = new byte[len];
        dis.readFully(data);
        dis.close();
        in.close();
```

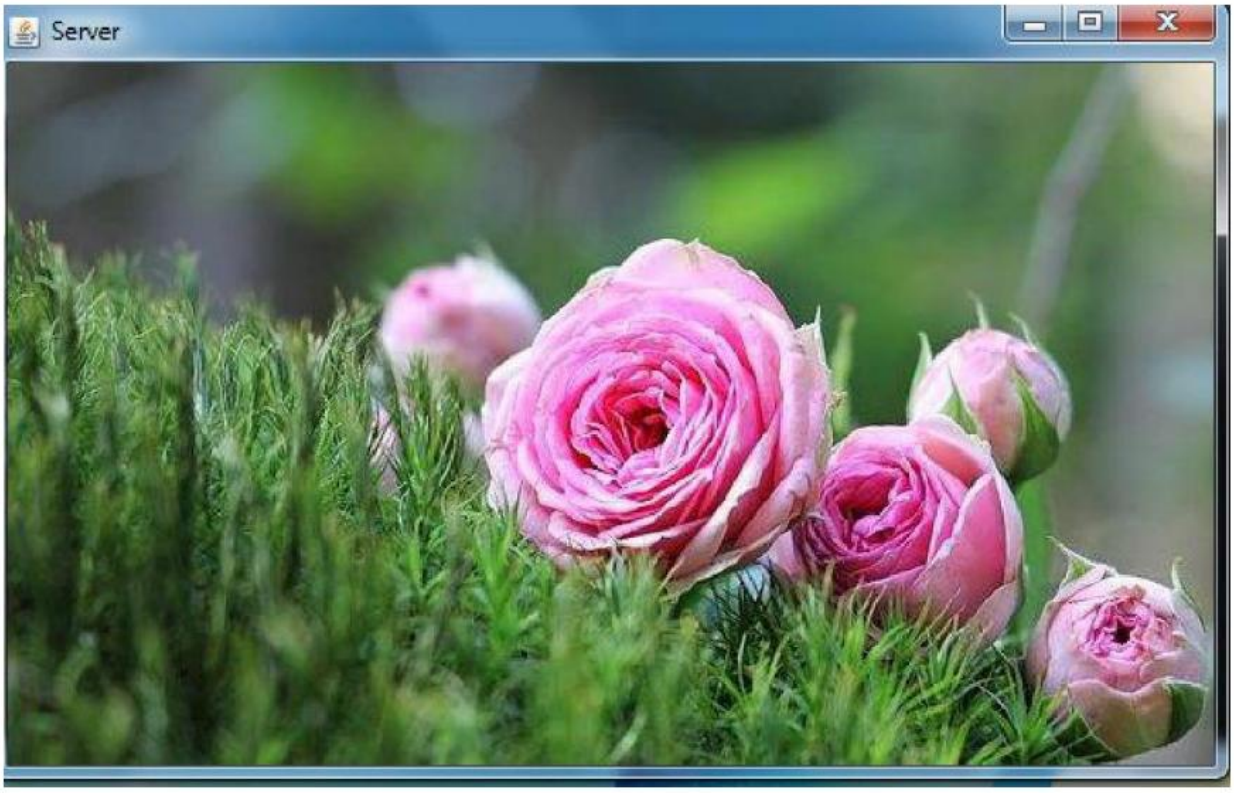
```
        InputStream ian = new ByteArrayInputStream(data);
        BufferedImage bImage = ImageIO.read(ian);

        JFrame f = new JFrame("Server");
        ImageIcon icon = new ImageIcon(bImage);
        JLabel l = new JLabel(icon);
        f.add(l);
        f.pack();
        f.setVisible(true);
    }
}
```

**OUTPUT:**






**RESULT:**

Thus, the HTTP socket for web page upload and download using Java was written and executed successfully.

**EX. NO.: 3(a)** **SOCKET PROGRAM FOR ECHO**

**Date:**

**AIM:**

To write a socket program for implementation of echo.

**ALGORITHM:**

**CLIENT SIDE**

1. Start the program.
2. Create a socket binding the server IP and port.
3. Send data to the server.
4. Receive and print the same data from server.
5. Close the socket.

**SERVER SIDE**

1. Start the program.
2. Create a server socket at a specified port.
3. Accept client connection.
4. Receive data from client.
5. Print and send back the same data to client.
6. Close the socket.

**PROGRAM:**

**Echo Client (eclient.java)**

```
import java.io.*;
import java.net.*;
public class eclient {
   public static void main(String args[]) {
```

```
        Socket c = null;
        String line;
        DataInputStream is, is1;
        PrintStream os;
        try {
            c = new Socket("localhost", 8080);
        } catch(IOException e) {
            System.out.println(e);
        }
        try {
            os = new PrintStream(c.getOutputStream());
            is = new DataInputStream(System.in);
            is1 = new DataInputStream(c.getInputStream());
            do {
                System.out.println("client");
                line = is.readLine();
                os.println(line);
                if(!line.equals("exit"))
                    System.out.println("server:" + is1.readLine());
            } while(!line.equals("exit"));
        } catch(IOException e) {
            System.out.println("socket closed");
        }
    }
}
```

**Echo Server (eserver.java)**

```
import java.io.*;
import java.net.*;

public class eserver {
   public static void main(String args[]) throws IOException {
      ServerSocket s = null;
      String line;
      DataInputStream is;
      PrintStream ps;
      Socket c = null;

      try {
         s = new ServerSocket(8080);
      } catch(IOException e) {
         System.out.println(e);
      }

      try {
         c = s.accept();
         is = new DataInputStream(c.getInputStream());
         ps = new PrintStream(c.getOutputStream());
         while(true) {
            line = is.readLine();
            System.out.println("msg received and sent back to client");
            ps.println(line);
         }
```

```
        } catch(IOException e) {
            System.out.println(e);
        }
    }
}
```

**OUTPUT:**

**CLIENT:**

Enter IP 127.0.0.1

CONNECTION ESTABLISHED

Enter data: SRM

Client received: SRM

**SERVER:**

CONNECTION ACCEPTED

Server received: SRM

**RESULT:**

The program for simulation of echo server was written and executed successfully.

**EX. NO.: 3(b)** **CLIENT-SERVER APPLICATION FOR CHAT**

**Date:**

**AIM:**

To write a client-server application for chat using TCP.

**ALGORITHM:**

**CLIENT SIDE**

1. Start the program.
2. Include necessary Java packages.
3. Create a socket to connect to the server.
4. Send data to the server.
5. Receive data from the server.
6. End communication on exit.

**SERVER SIDE**

1. Start the program.
2. Include necessary Java packages.
3. Create a socket to accept client connections.
4. Receive and send data to client.
5. End communication on exit.

**PROGRAM:**

**TCPserver1.java**

```
import java.net.*;
import java.io.*;
public class TCPserver1 {
   public static void main(String arg[]) {
      ServerSocket s = null;
      String line;
```

```
        DataInputStream is=null, is1=null;
        PrintStream os=null;
        Socket c=null;
        try {
            s = new ServerSocket(9999);
        } catch(IOException e) {
            System.out.println(e);
        }
        try {
            c = s.accept();
            is = new DataInputStream(c.getInputStream());
            is1 = new DataInputStream(System.in);
            os = new PrintStream(c.getOutputStream());
            do {
                line = is.readLine();
                System.out.println("Client:" + line);
                System.out.print("Server: ");
                line = is1.readLine();
                os.println(line);
            } while(!line.equalsIgnoreCase("quit"));
            is.close();
            os.close();
        } catch(IOException e) {
            System.out.println(e);
        }
    }
}
```

**TCPclient1.java**

```
import java.net.*;
import java.io.*;

public class TCPclient1 {
   public static void main(String arg[]) {
      Socket c = null;
      String line;
      DataInputStream is, is1;
      PrintStream os;

      try {
         c = new Socket("10.0.200.36", 9999);
      } catch(IOException e) {
         System.out.println(e);
      }

      try {
         os = new PrintStream(c.getOutputStream());
         is = new DataInputStream(System.in);
         is1 = new DataInputStream(c.getInputStream());
         do {
            System.out.print("Client: ");
            line = is.readLine();
            os.println(line);
            System.out.println("Server: " + is1.readLine());
         } while(!line.equalsIgnoreCase("quit"));
```

```
            is1.close();
            os.close();
        } catch(IOException e) {
            System.out.println("Socket Closed! Message Passing is over");
        }
    }
}
```

**OUTPUT:**

**SERVER:**

Client: Hai Server

Server: Hai Client

Client: How are you

Server: Fine

Client: quit

Server: quit

**CLIENT:**

Client: Hai Server

Server: Hai Client

Client: How are you

Server: Fine

Client: quit

Server: quit

**RESULT:**

The client-server chat application using TCP/IP was written and executed successfully.

**EX. NO.: 3(c)** **FILE TRANSFER IN CLIENT & SERVER**

**Date:**

**AIM:**

To perform file transfer in Client & Server using TCP/IP.

**ALGORITHM:**

**CLIENT SIDE**

1. Start the program.
2. Establish connection with the server.
3. Implement requests to get or send a file.
4. Receive approval from server and perform file transfer.

**SERVER SIDE**

1. Start the program.
2. Create a server socket to listen on a port.
3. Send the file for 'get' request.
4. Receive and store the file for 'put' request.
5. Stop upon client request.

**PROGRAM:**

**FileClient.java**

```
import java.net.*;
import java.io.*;
public class FileClient {
   public static void main(String[] args) throws IOException {
      int filesize = 6022386;
      int bytesRead;
```

```
        int current = 0;
        Socket sock = new Socket("127.0.0.1", 13267);
        System.out.println("Connecting...");
        byte[] mybytearray = new byte[filesize];
        InputStream is = sock.getInputStream();
        FileOutputStream fos = new FileOutputStream("source-copy.pdf");
        BufferedOutputStream bos = new BufferedOutputStream(fos);
        bytesRead = is.read(mybytearray, 0, mybytearray.length);
        current = bytesRead;
        do {
           bytesRead = is.read(mybytearray, current, (mybytearray.length - current));
           if(bytesRead >= 0) current += bytesRead;
        } while(bytesRead > -1);
        bos.write(mybytearray, 0, current);
        bos.flush();
        bos.close();
        sock.close();
    }
}
```

**FileServer.java**

```
import java.net.*;
import java.io.*;
public class FileServer {
    public static void main(String[] args) throws IOException {
        ServerSocket servsock = new ServerSocket(13267);
        while(true) {
           System.out.println("Waiting...");
```

```
            Socket sock = servsock.accept();
            System.out.println("Accepted connection: " + sock);

            File myFile = new File("source.pdf");
            byte[] mybytearray = new byte[(int) myFile.length()];
            FileInputStream fis = new FileInputStream(myFile);
            BufferedInputStream bis = new BufferedInputStream(fis);
            bis.read(mybytearray, 0, mybytearray.length);
            OutputStream os = sock.getOutputStream();
            System.out.println("Sending...");
            os.write(mybytearray, 0, mybytearray.length);
            os.flush();
            sock.close();
        }
    }
}
```

**OUTPUT:**

**SERVER:**

Waiting for clients...

Connection Established

Client wants file: network.txt

**CLIENT:**

Connecting...

File received: network.txt

Content displayed successfully.

**RESULT:**

File transfer operation between client and server using TCP/IP was written and executed successfully.

**EX. NO.: 4 IMPLEMENTATION OF DNS USING UDP SOCKETS**

**Date:**

**AIM**

To implement a DNS server and client in Java using UDP sockets.

**OUTCOME**

Students become proficient in writing socket programs for DNS in Java using UDP sockets.

**ALGORITHM**

**Server:**

1. Define an array of hostnames and corresponding IP addresses.
2. Create a datagram socket.
3. Create a datagram packet to receive the client's request.
4. Read the domain name from the client to be resolved.
5. Lookup the hostname in the array.
6. If found, retrieve the corresponding IP address.
7. Construct a datagram packet to send the response back to the client.
8. Repeat for multiple client requests.
9. Close the socket.
10. Stop.

**Client:**

1. Create a datagram socket.
2. Get the domain name from the user.
3. Send the domain name to the server via a datagram packet.
4. Receive the response packet from the server.
5. If the response contains an IP address, display it; otherwise display "Domain does not exist."
6. Close the socket.

**PROGRAM**

**UDP DNS Server — Udpdnsserver.java**

```
import java.io.*;
import java.net.*;

public class Udpdnsserver {
   private static int indexOf(String[] array, String str) {
      str = str.trim();
      for (int i = 0; i < array.length; i++) {
         if (array[i].equals(str))
            return i;
      }
      return -1;
   }

   public static void main(String arg[]) throws IOException {
      String[] hosts = {"yahoo.com", "gmail.com", "cricinfo.com", "facebook.com"};
      String[] ip = {"68.180.206.184", "209.85.148.19", "80.168.92.140", "69.63.189.16"};
      System.out.println("DNS Server Started (Press Ctrl + C to Quit)");

      DatagramSocket serversocket = new DatagramSocket(1362);
      byte[] senddata = new byte[1024];
      byte[] receivedata = new byte[1024];

      while (true) {
         DatagramPacket recvpack = new DatagramPacket(receivedata, receivedata.length);
         serversocket.receive(recvpack);
```

```
        String domain = new String(recvpack.getData()).trim();

        InetAddress ipaddress = recvpack.getAddress();
        int port = recvpack.getPort();

        System.out.println("Request for host: " + domain);
        String response;

        if (indexOf(hosts, domain) != -1)
           response = ip[indexOf(hosts, domain)];
        else
           response = "Host Not Found";

        senddata = response.getBytes();
        DatagramPacket pack = new DatagramPacket(senddata, senddata.length, ipaddress, port);
        serversocket.send(pack);
      }
   }
}
```

**UDP DNS Client — Udpdnsclient.java**

```
import java.io.*;
import java.net.*;

public class Udpdnsclient {
   public static void main(String args[]) throws IOException {
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
      DatagramSocket clientsocket = new DatagramSocket();
```

```
        InetAddress ipaddress;
        if (args.length == 0)
           ipaddress = InetAddress.getLocalHost();
        else
           ipaddress = InetAddress.getByName(args[0]);

        byte[] senddata = new byte[1024];
        byte[] receivedata = new byte[1024];
        int portaddr = 1362;

        System.out.print("Enter the hostname: ");
        String sentence = br.readLine();
        senddata = sentence.getBytes();

        DatagramPacket pack = new DatagramPacket(senddata, senddata.length, ipaddress,
portaddr);
        clientsocket.send(pack);

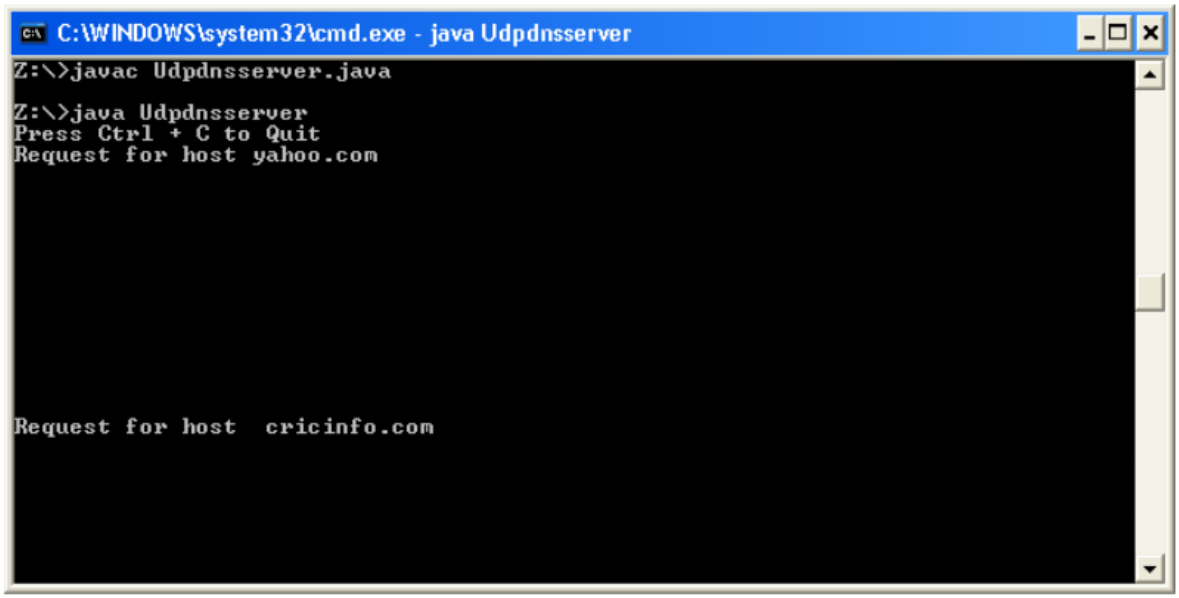
        DatagramPacket recvpack = new DatagramPacket(receivedata, receivedata.length);
        clientsocket.receive(recvpack);

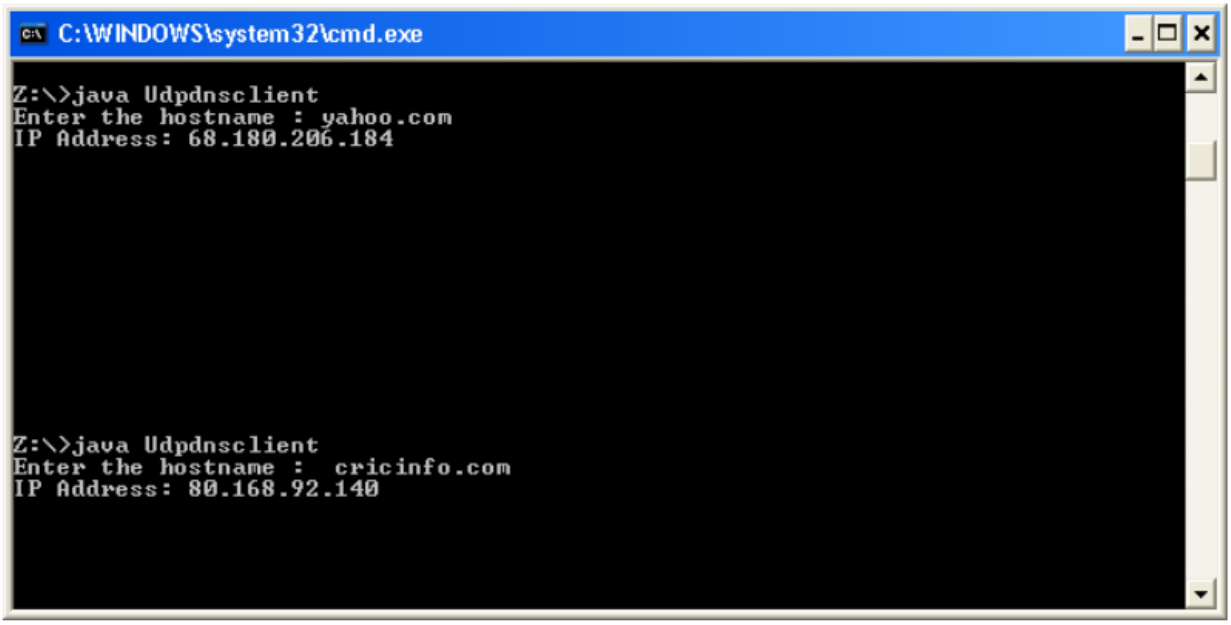
        String modified = new String(recvpack.getData());
        System.out.println("IP Address: " + modified.trim());
        clientsocket.close();
    }
}
```

**OUTPUT**

Enter the hostname: gmail.com

IP Address: 209.85.148.19




**RESULT**

Thus, the domain name requested by the client was successfully resolved into its corresponding IP address using UDP socket programming in Java.

**EX. NO.: 5(a)** **IMPLEMENTATION OF ARP PROTOCOL**

**Date:**

## AIM

To acquire knowledge on ARP and UDP concepts and implement ARP protocol simulation using Java.

## OUTCOME

Students gain practical knowledge in understanding and implementing ARP using Java sockets.

## ALGORITHM

**Server:**

1. Create a server socket.
2. Accept client connection.
3. Read IP address from the client.
4. Compare with the configuration file (local system info).
5. If a match is found, send the MAC address.
6. Stop.

**Client:**

1. Create a socket.
2. Send IP address to the server.
3. Receive the MAC address.
4. If received, display it; otherwise show “Host not found.”
5. Stop.

**PROGRAM**

**ARP Server — Arpserver.java**

```
import java.io.*;
import java.net.*;


class Arpserver {
   public static void main(String args[]) throws IOException {
      try {
         ServerSocket soc = new ServerSocket(2500);
         System.out.println("Server started...");
         Socket client = soc.accept();


         PrintStream ps = new PrintStream(client.getOutputStream());
         BufferedReader br = new BufferedReader(new
InputStreamReader(client.getInputStream()));


         Runtime r = Runtime.getRuntime();
         Process p = r.exec("ifconfig eth0");
         BufferedReader pin = new BufferedReader(new
InputStreamReader(p.getInputStream()));


         String str, haddr = "";
         String ipaddr = br.readLine();
         int flag = 0;


         while ((str = pin.readLine()) != null) {
            if (str.contains("HWaddr")) {
               haddr = str.substring(str.indexOf("HWaddr") + 7).trim();
```

```
            } else if (str.contains(ipaddr)) {
                flag = 1;
            }
        }

        if (flag == 1)
            ps.println(haddr);
        else
            ps.println("Host does not exist");

        ps.close();
        br.close();
        pin.close();
        client.close();
        soc.close();
      } catch (IOException io) {
        System.err.println("Exception: " + io);
      }
   }
}
```

**ARP Client — Arpclient.java**

```
import java.io.*;
import java.net.*;

class Arpclient {
   public static void main(String args[]) {
      try {
```

```
        Socket client = new Socket("localhost", 2500);
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintStream ps = new PrintStream(client.getOutputStream());
        BufferedReader sin = new BufferedReader(new
InputStreamReader(client.getInputStream()));

        System.out.print("Enter the IP address: ");
        String ipaddr = br.readLine();
        ps.println(ipaddr);

        String haddr = sin.readLine();
        System.out.println("Physical Address: " + haddr);

        ps.close();
        br.close();
        client.close();
      } catch (IOException io) {
        System.err.println(io);
      }
   }
}
```

**OUTPUT**

Enter the IP address: 192.168.1.5

Physical Address: 00:0C:29:5D:8E:9B

```
siva@siva-virtual-machine:~/Downloads$ javac arpserver.java
siva@siva-virtual-machine:~/Downloads$ java arpserver
Server started
eth0 Link encap:Ethernet HWaddr 00:0c:29:6d:92:62
inet addr:192.168.240.129 Bcast:192.168.240.255 Mask:255.255.255.0
inet6 addr: fe80::20c:29ff:fe6d:9262/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:16203 errors:0 dropped:0 overruns:0 frame:0
TX packets:18505 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:10879053 (10.8 MB) TX bytes:2278999 (2.2 MB)
Interrupt:19 Base address:0x2000
siva@siva-virtual-machine:~/Downloads$
```

```
iva@siva-virtual-machine:~/Downloads$ java arpclient
nter the IP address : 192.168.240.1
hysical Address 00:0c:29:6d:92:62
iva@siva-virtual-machine:~/Downloads$
```

**RESULT**

Thus, the Java program to simulate the ARP protocol was executed successfully.

**EX. NO.: 5(b)** **SIMULATION OF RARP PROTOCOL**

**Date:**

## AIM

To simulate RARP protocol using Java language.

## OUTCOME

Students gain knowledge in understanding and simulating RARP protocol using Java.

## ALGORITHM

**Server:**

1. Create a server socket.
2. Accept client connection.
3. Read MAC address from client.
4. Compare with configuration file.
5. If a match is found, send the IP address.
6. Stop.

**Client:**

1. Create a socket.
2. Send MAC address to the server.
3. Receive IP address.
4. Display the IP address if found, else show “Host not found.”
5. Stop.

**PROGRAM**

**RARP Server — Rarpserver.java**

```
import java.io.*;
import java.net.*;


class Rarpserver {
   public static void main(String args[]) throws IOException {
      try {
         ServerSocket soc = new ServerSocket(5000);
         System.out.println("Server started...");
         Socket client = soc.accept();


         PrintStream ps = new PrintStream(client.getOutputStream());
         BufferedReader br = new BufferedReader(new
InputStreamReader(client.getInputStream()));


         Runtime r = Runtime.getRuntime();
         Process p = r.exec("ifconfig eth0");
         BufferedReader pin = new BufferedReader(new
InputStreamReader(p.getInputStream()));


         String str, ipaddr = "";
         String haddr = br.readLine();
         int flag = 0;


         while ((str = pin.readLine()) != null) {
            if (str.contains(haddr)) {
               flag = 1;
```

```
            } else if (str.contains("inet addr")) {
                int pos = str.indexOf("inet addr:") + 10;
                ipaddr = str.substring(pos, str.indexOf(" ", pos));
            }
        }

        if (flag == 1)
            ps.println(ipaddr);
        else
            ps.println("Host not found");

        ps.close();
        br.close();
        pin.close();
        client.close();
        soc.close();
      } catch (IOException io) {
        System.err.println("Exception: " + io);
      }
   }
}
```

**RARP Client — Rarpclient.java**

```
import java.io.*;
import java.net.*;

class Rarpclient {
   public static void main(String args[]) {
```

```
        try {
            Socket client = new Socket("localhost", 5000);
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            PrintStream ps = new PrintStream(client.getOutputStream());
            BufferedReader sin = new BufferedReader(new
InputStreamReader(client.getInputStream()));

            System.out.print("Enter the physical address: ");
            String haddr = br.readLine();
            ps.println(haddr);

            String ipaddr = sin.readLine();
            System.out.println("Logical Address: " + ipaddr);

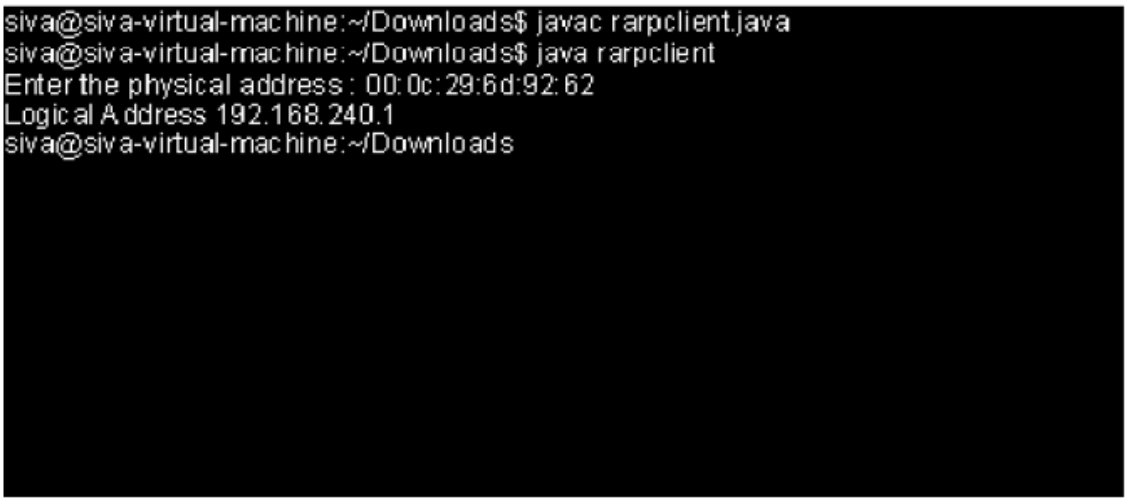
            ps.close();
            br.close();
            client.close();
        } catch (IOException io) {
            System.err.println(io);
        }
    }
}
```

**OUTPUT**

Enter the physical address: 00:0C:29:5D:8E:9B

Logical Address: 192.168.1.5




**RESULT**

Thus, the Java program to simulate the RARP protocol was executed successfully.

**EX. NO.: 6(a)** **STUDY OF NETWORK SIMULATOR (NS2)**

**Date:**

**INTRODUCTION:**

Network Simulator (Version 2), widely known as **NS2**, is an event-driven simulation tool used for studying the dynamic nature of communication networks. It allows simulation of both **wired and wireless network protocols** such as routing algorithms, TCP, and UDP.

NS2 provides users with a method to specify network protocols and simulate their behaviors. Its flexibility and modular nature have made it a popular tool in the research community since 1989.

Initially developed from the **REAL network simulator** by the University of California and Cornell University, NS2 later received support from **DARPA's VINT project** and **NSF**, with contributions from researchers and developers worldwide.

**BASIC ARCHITECTURE:**

NS2 uses two key languages — **C++** and **OTcl (Object-oriented Tcl)**.

- **C++** defines the internal mechanisms (backend).
- **OTcl** sets up the simulation and schedules events (frontend).

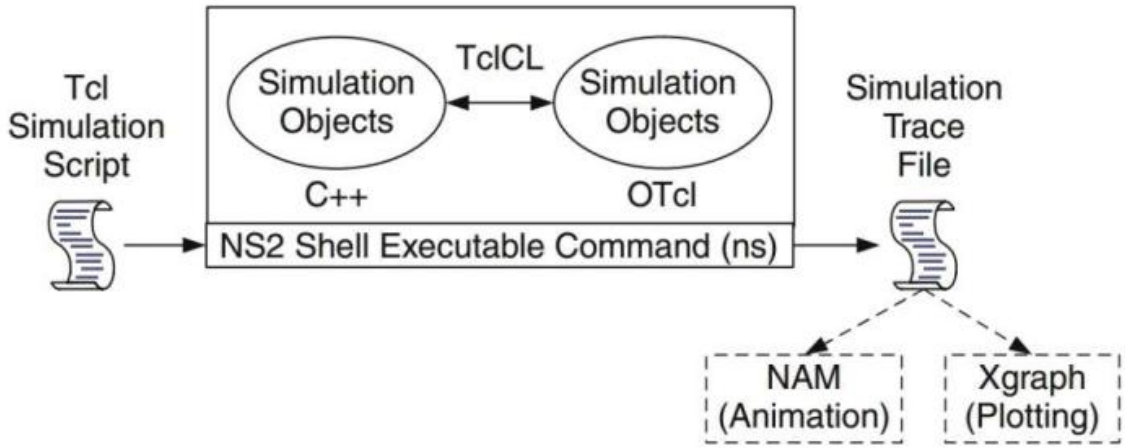



**Fig. 2.1.** Basic architecture of NS.

They are linked together using **TclCL**. The user interacts mainly through OTcl scripts, while the actual network behaviors are implemented in C++.

A simulation script written in OTcl is executed using the command:

ns <filename>.tcl

This generates trace files for analysis using **NAM (Network Animator)** or **XGraph**.

**CONCEPT OVERVIEW:**

NS2 uses **two languages** because the simulator handles both:

- **Low-level protocol operations** (efficiently handled in C++).
- **High-level experiment configuration** (flexibly done in OTcl).

This dual approach allows high performance and rapid experimentation.

**SIMULATOR INITIALIZATION:**

When a simulation object is created in Tcl, the initialization performs:

- Packet format creation.
- Scheduler setup (default: calendar queue).
- Creation of a "null agent" for discarded packets.

**SCHEDULERS AND EVENTS:**

NS2 is an **event-driven simulator**.

- Each event is executed sequentially based on scheduled time.
- Default scheduler type: **calendar queue**.
- Supports single-threaded execution and first-scheduled–first-dispatched order.

**NODE BASICS:**

Nodes are basic network elements created using:

set ns [new Simulator]

set n0 [$ns node]

Each node includes classifiers that route packets to appropriate destinations.

**BASIC COMMANDS IN NS2:**

| **Command** | **Description** |
|---|---|
| set ns [new Simulator] | Creates a simulator object. |
| $ns node | Creates a network node. |
| $ns duplex-link n1 n2 bw delay DropTail | Connects two nodes with a link. |
| $ns queue-limit n1 n2 value | Sets queue limit. |

| | |
|---|---|
| $ns at time "command" | Schedules a command at specific time. |
| $ns run | Starts the simulation. |
| $ns trace-all file | Records trace information. |
| $ns namtrace-all file | Creates NAM trace file. |

**INSTALLATION PROCEDURE (NS2.34):**

1. Download NS2.34 package:
    1. http://nsnam.isi.edu/nsnam/index.php/Downloading_and_installing_ns-2

2. Extract and install:
    1. tar -xzf ns-allinone-2.34.tar.gz
    2. cd ns-allinone-2.34
    3. ./install

3. After successful installation, set environment variables:
    1. export LD_LIBRARY_PATH=/home/user/ns-allinone-2.34/lib
    2. export PATH=$PATH:/home/user/ns-allinone-2.34/bin
    3. export TCL_LIBRARY=/home/user/ns-allinone-2.34/tcl8.4.11/library

4. Validate installation:
    1. cd ns-2.34
    2. ./validate

**RESULT:**

Thus, the study of **Network Simulator (NS2)** was successfully completed and its architecture, installation, and working principles were understood.

**EX. NO: 6(b) SIMULATION OF CONGESTION CONTROL ALGORITHM USING NS2**

**Date:**

**AIM:**

To acquire TCL scripting skills and implement congestion control algorithms using NS2 simulator.

**OUTCOME:**

Students gain expertise in writing TCL scripts to simulate **Stop-and-Wait** and **Go-Back-N** congestion control protocols using NS2.

**ALGORITHM:**

1. Start the program.
2. Create the trace and NAM files.
3. Set up the network topology.
4. Create nodes and establish duplex links.
5. Attach agents and queues (DropTail).
6. Configure initial positions of nodes.
7. Setup TCP connection between nodes.
8. Define simulation window positions.
9. Schedule events and end the simulation.
10. Stop the NAM and simulation.

**PROGRAM 1: STOP-AND-WAIT PROTOCOL**

```
set ns [new Simulator]

set n0 [$ns node]

set n1 [$ns node]

set n2 [$ns node]
```

```
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]

# Node colors
$n0 color "purple"
$n1 color "purple"
$n2 color "violet"
$n3 color "violet"
$n4 color "chocolate"
$n5 color "chocolate"

# Trace files
set f [open stopwait.tr w]
$ns trace-all $f
set nf [open stopwait.nam w]
$ns namtrace-all $nf

# Labels
$ns at 0.0 "$n0 label SYS0"
$ns at 0.0 "$n1 label SYS1"
$ns at 0.0 "$n2 label SYS2"
$ns at 0.0 "$n3 label SYS3"
$ns at 0.0 "$n4 label SYS4"
$ns at 0.0 "$n5 label SYS5"
```

```
# Links
$ns duplex-link $n0 $n2 0.2Mb 20ms DropTail
$ns duplex-link $n1 $n2 0.2Mb 20ms DropTail
$ns duplex-link $n2 $n3 0.2Mb 20ms DropTail
$ns duplex-link $n3 $n4 0.2Mb 20ms DropTail
$ns duplex-link $n3 $n5 0.2Mb 20ms DropTail

# Orientation and queue limit
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right
$ns queue-limit $n0 $n2 10

# TCP Agent
set tcp [new Agent/TCP]
$tcp set window 1
$tcp set maxcwnd 1
$tcp set fid 1
$ns attach-agent $n0 $tcp

set sink [new Agent/TCPSink]
$ns attach-agent $n5 $sink
$ns connect $tcp $sink

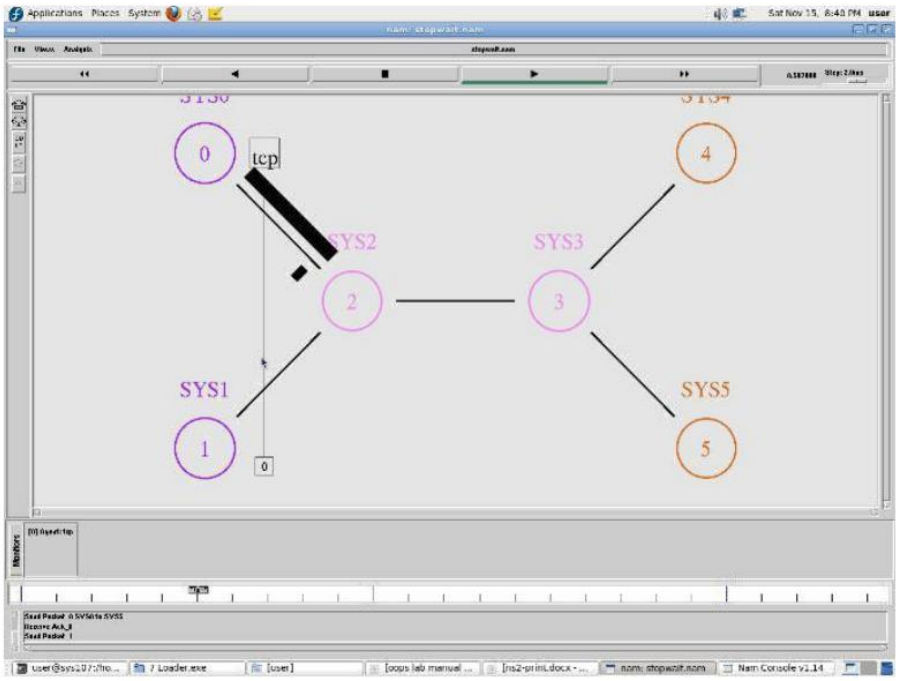
# FTP Application
set ftp [new Application/FTP]
$ftp attach-agent $tcp
```

```
# Simulation events
$ns at 0.1 "$ftp start"
$ns at 2.5 "finish"

proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam stopwait.nam &
    exit 0
}
$ns run
```

**OUTPUT:**

**STOP WAIT PROTOCOL**



**PROGRAM 2: GO-BACK-N PROTOCOL**

```
set ns [new Simulator]
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]

# Colors and labels
$n0 color "purple"
$n1 color "purple"
$n2 color "violet"
$n3 color "violet"
$n4 color "chocolate"
$n5 color "chocolate"

# Trace files
set f [open goback.tr w]
$ns trace-all $f
set nf [open goback.nam w]
$ns namtrace-all $nf

# Links
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
$ns duplex-link $n3 $n5 1Mb 10ms DropTail

# TCP Configuration
set tcp [new Agent/TCP]
$tcp set fid 1
$ns attach-agent $n1 $tcp

set sink [new Agent/TCPSink]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink

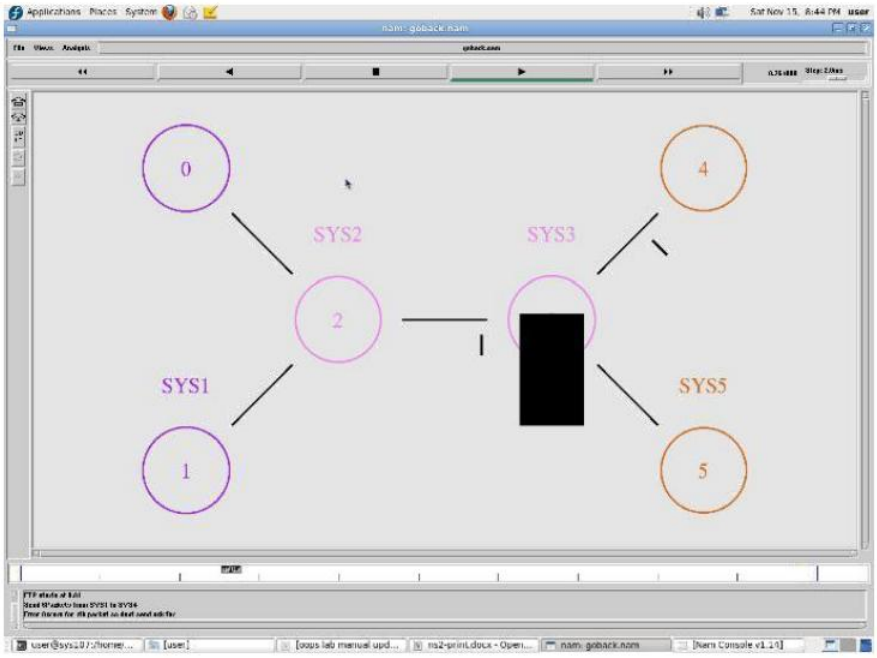
set ftp [new Application/FTP]
$ftp attach-agent $tcp

# Simulation events
$ns at 0.05 "$ftp start"
$ns at 1.5 "finish"

proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam goback.nam &
    exit 0
}
$ns run
```

**OUTPUT:**

**GO BACK N protocol**




**RESULT:**

Thus, the TCL programs were executed successfully to simulate the **Stop-and-Wait** and **Go-Back-N** congestion control protocols using **NS2 simulator**.

**EX. NO.: 7 STUDY OF TCP/UDP PERFORMANCE USING SIMULATION TOOL**

**Date:**

**AIM:**

To simulate **TCP/UDP performance** using NS2 simulation tool.

**OUTCOME:**

Students gain knowledge to compare and analyze **TCP and UDP performance** using NS2.

**PROGRAM:**

```
# Create Simulator
set ns [new Simulator]

# Define colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red

# Open trace files
set file1 [open out.tr w]
set winfile [open WinFile w]
$ns trace-all $file1

# Open NAM trace file
set file2 [open out.nam w]
$ns namtrace-all $file2
```

```
# Define finish procedure
proc finish {} {
    global ns file1 file2
    $ns flush-trace
    close $file1
    close $file2
    exec nam out.nam &
    exit 0
}

# Create nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]

# Node color and shape
$n1 color red
$n1 shape box

# Create links between nodes
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns simplex-link $n2 $n3 0.3Mb 100ms DropTail
$ns simplex-link $n3 $n2 0.3Mb 100ms DropTail
```

```
# Create LAN
set lan [$ns newLan "$n3 $n4 $n5" 0.5Mb 40ms LL Queue/DropTail MAC/Csma/Cd Channel]

# Setup TCP connection
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$tcp set window_ 8000
$tcp set packetSize_ 552

# Setup FTP over TCP
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP

# Setup UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n5 $null
$ns connect $udp $null
$udp set fid_ 2
```

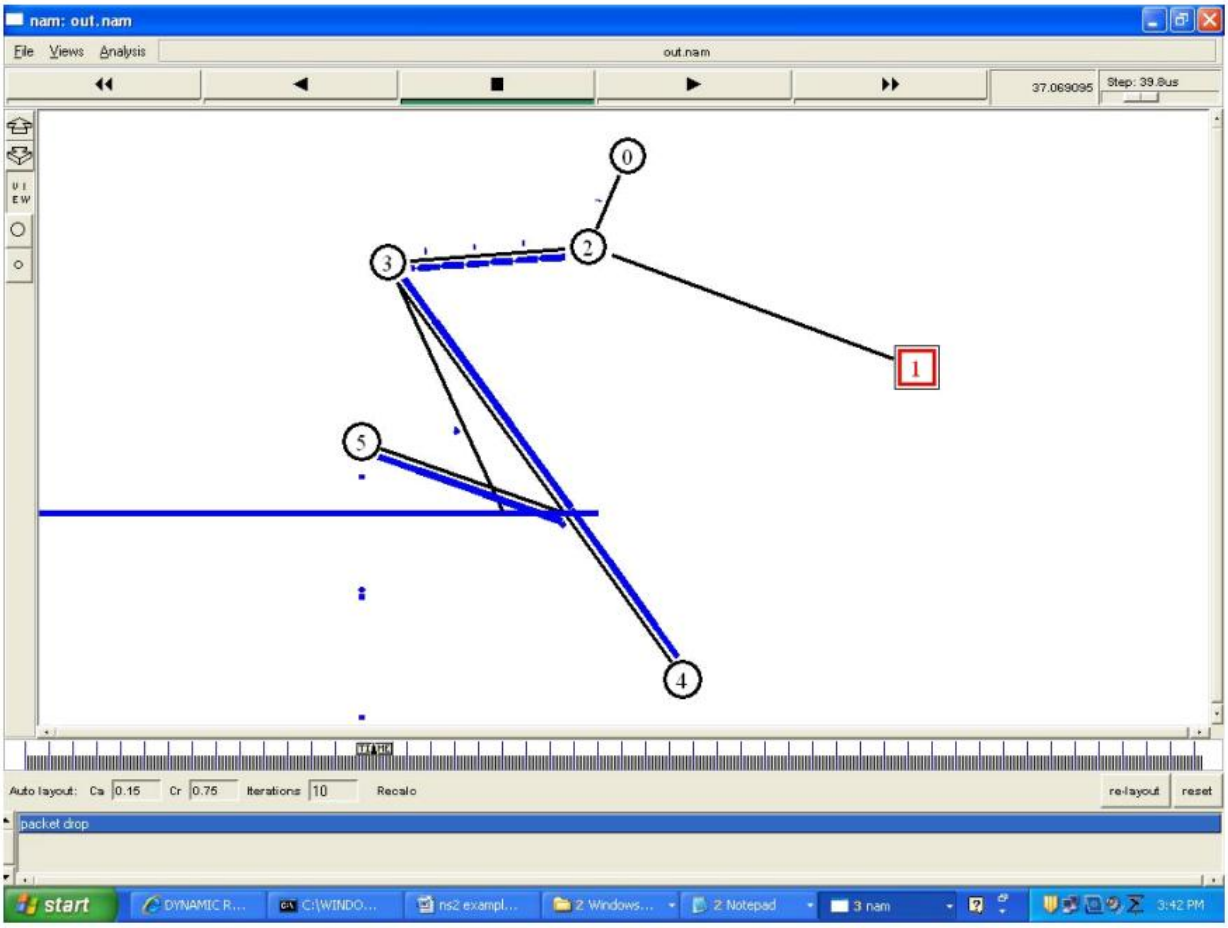
```
# Setup CBR over UDP
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 0.01mb
$cbr set random_ false

# Start and stop times
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 124.0 "$ftp stop"
$ns at 124.5 "$cbr stop"

# Window plotting procedure
proc plotWindow {tcpSource file} {
   global ns
   set time 0.1
   set now [$ns now]
   set cwnd [$tcpSource set cwnd_]
   puts $file "$now $cwnd"
   $ns at [expr $now+$time] "plotWindow $tcpSource $file"
}
$ns at 0.1 "plotWindow $tcp $winfile"
$ns at 5 "$ns trace-annotate \"packet drop\""
$ns at 125.0 "finish"
$ns run
```

**OUTPUT:**

The simulation runs successfully in **NAM window**, showing **TCP and UDP traffic performance** between nodes.




**RESULT:**

Thus, the TCL script to simulate **TCP/UDP performance** using **NS2 simulator** was executed successfully.

**EX. NO.: 8(a) SIMULATION OF LINK STATE ROUTING PROTOCOL USING NS2**

**Date:**

**AIM:**

To simulate the **Link State Routing Protocol** using NS2.

**OUTCOME:**

Students become proficient in writing TCL scripts to simulate the **Link State Routing Protocol** using NS2 simulator.

**PROGRAM:**

```
set ns [new Simulator]
set nr [open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf

proc finish { } {
   global ns nr nf
   $ns flush-trace
   close $nf
   close $nr
   exec nam thro.nam &
   exit 0
}

for { set i 0 } { $i < 12} { incr i 1 } {
```

```
    set n($i) [$ns node]
}

for {set i 0} {$i < 8} {incr i} {
    $ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail
}

$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail

# Traffic setup
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0

set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
```

```
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0

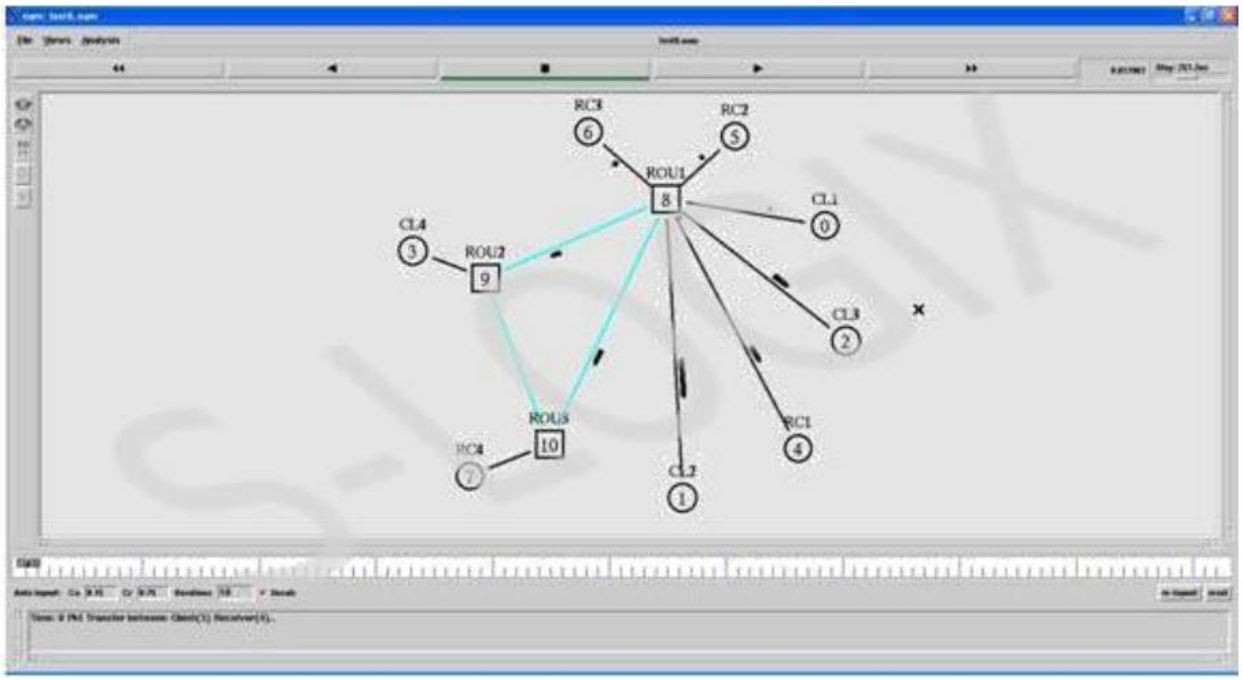
$ns rtproto LS
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)

$udp0 set fid_ 1
$udp1 set fid_ 2
$ns color 1 Red
$ns color 2 Green

$ns at 1.0 "$cbr0 start"
$ns at 2.0 "$cbr1 start"
$ns at 45 "finish"
$ns run
```

**OUTPUT:**

The **Link State Routing Protocol** is simulated successfully in **NAM**, showing routing updates and packet transmission among nodes.




**RESULT:**

Thus, the TCL program for **Link State Routing Protocol** was executed successfully using NS2 simulator.

**EX. NO.: 8(b) SIMULATION OF DISTANCE VECTOR ROUTING PROTOCOL USING NS2**

**Date:**

**AIM:**

To simulate the **Distance Vector Routing Protocol** using NS2.

**OUTCOME:**

Students gain expertise in writing TCL scripts to simulate **Distance Vector Routing Protocol** in NS2.

**PROGRAM:**

```
set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set tr [open out.tr w]
$ns trace-all $tr

proc finish {} {
   global nf ns tr
   $ns flush-trace
   close $tr
   exec nam out.nam &
   exit 0
}

# Node creation
set n0 [$ns node]
```

```
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

# Link setup
$ns duplex-link $n0 $n1 10Mb 10ms DropTail
$ns duplex-link $n1 $n3 10Mb 10ms DropTail
$ns duplex-link $n2 $n1 10Mb 10ms DropTail
$ns duplex-link-op $n0 $n1 orient right-down
$ns duplex-link-op $n1 $n3 orient right
$ns duplex-link-op $n2 $n1 orient right-up

# TCP and UDP setup
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
set ftp [new Application/FTP]
$ftp attach-agent $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
set udp [new Agent/UDP]
$ns attach-agent $n2 $udp
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
```

```
# Connections

$ns connect $tcp $sink

$ns connect $udp $null

# Link failures

$ns rtmodel-at 1.0 down $n1 $n3

$ns rtmodel-at 2.0 up $n1 $n3

# Protocol setup

$ns rtproto DV

$ns at 0.0 "$ftp start"

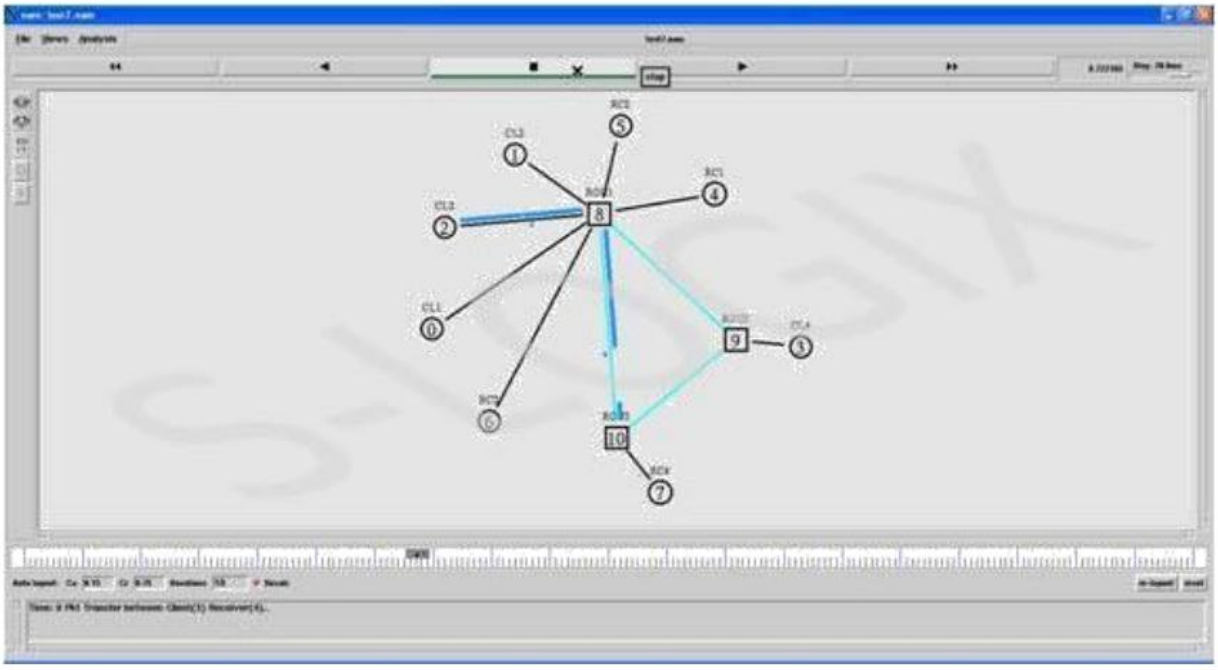
$ns at 0.0 "$cbr start"

$ns at 5.0 "finish"

$ns run
```

**OUTPUT:**

The simulation successfully shows **Distance Vector routing behavior**, with dynamic route updates and traffic flow.



**RESULT:**

Thus, the TCL program to simulate **Distance Vector Routing Protocol** using NS2 was executed successfully.

**EX. NO.: 9 PERFORMANCE ANALYSIS OF ROUTING PROTOCOL USING NS2**

**Date:**

**AIM:**

To simulate Distance Vector Multicast Routing Protocol in NS2.

**PROGRAM:**

```
#define options
set val(chan) Channel/WirelessChannel ;
set val(prop) Propogation/TwoRayGround ;
set val(netif) Phy/WirelessPhy ;
set val(mac) Mac/802_11 ;
set val(ifq) Queue/DropTail/PriQueue ;
set val(ll) LL ;
set val(ant) Antenna/OmniAntenna ;
set val(ifqlen) 50 ;
set val(nn) 3 ;
set val(rp) DSDV ;
set val(x) 500;
set val(y) 400;
set val(stop) 150;

set ns [new Simulator]
set tracefd [open simple.tr w]
set windowVsTime2 [open win.tr w]
set namtrace [open simwrls.nam w]
$ns trace-all $tracefd
```

```
$ns namtrace-all-wireless $namtrace $val(x) $val(y)

#set up topography object
set topo [new Topography]
$topo load_flatgrid $val(x) $val(y)
create-god $val(nn)

#configure the nodes
$ns node-config -adhocRouting DSR \
-llType LL \
-macType Mac/802_11 \
-ifqType Queue/DropTail/PriQueue \
-ifqLen 50 \
-antType Antenna/OmniAntenna \
-propType Propagation/TwoRayGround \
-phyType Phy/WirelessPhy \
-channelType Channel/WirelessChannel \
-topoInstance $topo \
-agentTrace ON \
-routerTrace ON \
-macTrace OFF \
-movementTrace ON

for {set i 0} {$i < $val(nn)} {incr i} {
   set node_($i) [$ns node]
}
```

```
#provide initial location of mobile nodes
$node_(0) set X_ 5.0
$node_(0) set Y_ 5.0
$node_(0) set Z_ 0.0
$node_(1) set X_ 490.0
$node_(1) set Y_ 285.0
$node_(1) set Z_ 0.0
$node_(2) set X_ 150.0
$node_(2) set Y_ 240.0
$node_(2) set Z_ 0.0

#generation of movements
$ns at 10.0 "$node_(0) setdest 250.0 250.0 3.0"
$ns at 15.0 "$node_(1) setdest 45.0 285.0 5.0"
$ns at 110.0 "$node_(0) setdest 480.0 300.0 5.0"

#set up TCP connection between node_(0) and node_(1)
set tcp [new Agent/TCP/Newreno]
$tcp set class_ 2
set sink [new Agent/TCPSink]
$ns attach-agent $node_(0) $tcp
$ns attach-agent $node_(1) $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ns at 10.0 "$ftp start"
```

```
#define node initial position in nam
for {set i 0} {$i < $val(nn)} {incr i} {
   $ns initial_node_pos $node_($i) 30
}

#telling nodes when the simulation ends
for {set i 0} {$i < $val(nn)} {incr i} {
   $ns at $val(stop) "$node_($i) reset";
}

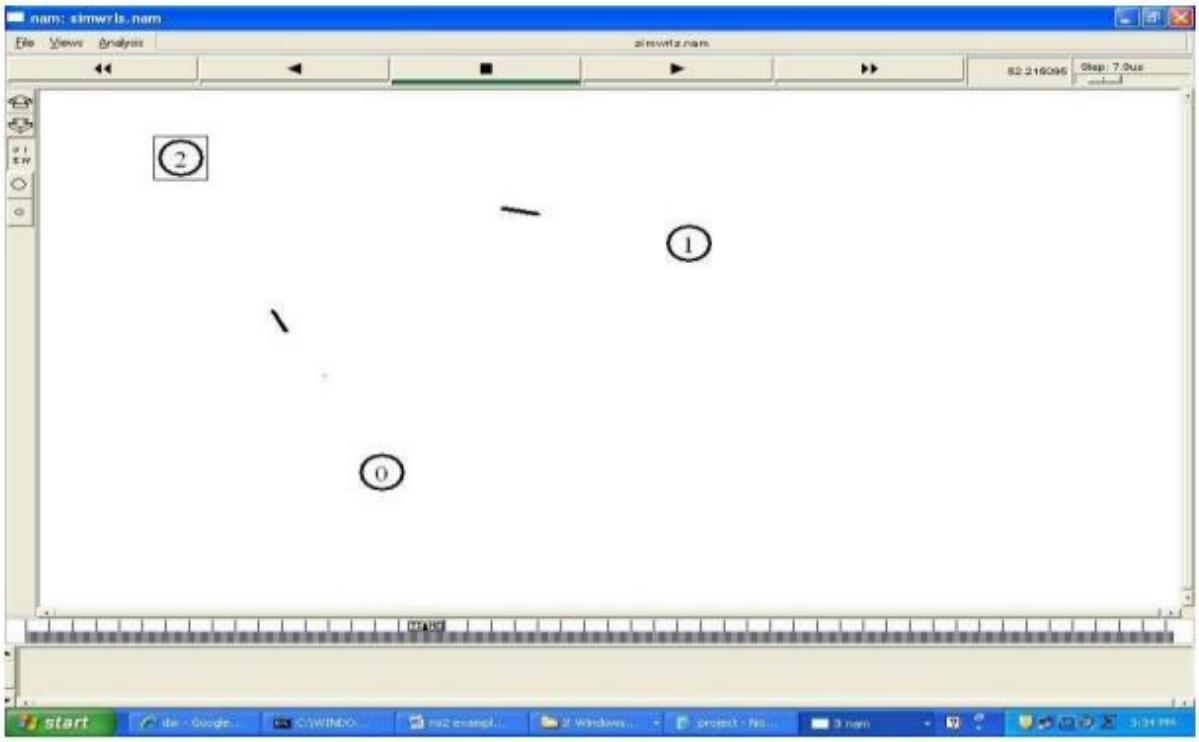
#ending nam and simulation
$ns at $val(stop) "$ns nam-end-wireless $val(stop)"
$ns at $val(stop) "stop"
$ns at 150.0 "puts \"end simulation\" ; $ns halt"

proc stop {} {
   global ns tracefd namtrace
   $ns flush-trace
   close $tracefd
   close $namtrace
   exec nam simwrls.nam &
}

$ns run
```

**OUTPUT:**

Simulation window showing node movement and data transmission between nodes using Distance Vector Multicast Routing Protocol.




**RESULT:**

Thus, the Distance Vector Multicast Routing Protocol was simulated successfully in NS2.

**EX. NO.: 10 SIMULATION OF ERROR CORRECTION CODES (CRC)**

**Date:**

**AIM:**

To implement error correction using CRC (Cyclic Redundancy Check).

**OUTCOME:**

Students gain knowledge in error detection and correction using cyclic redundancy check concepts.

**ALGORITHM:**

1. Start the program.
2. Create an object for the class process.
3. Get the number of bits to be sent.
4. Get the divisor value.
5. Generate the dividend value with appended bits.
6. Modify a bit to simulate an error in the sender side.
7. Check for error detection at the receiver side.
8. Stop the program.

**PROGRAM:**

```
import java.io.*;
import java.util.*;


class crc_gen {
   public static void main(String args[]) throws IOException {
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

```
int[] data, div, divisor, rem, crc;
int data_bits, divisor_bits, tot_length;

System.out.println("Enter number of data bits : ");
data_bits = Integer.parseInt(br.readLine());
data = new int[data_bits];
System.out.println("Enter data bits : ");
for (int i = 0; i < data_bits; i++)
    data[i] = Integer.parseInt(br.readLine());

System.out.println("Enter number of bits in divisor : ");
divisor_bits = Integer.parseInt(br.readLine());
divisor = new int[divisor_bits];
System.out.println("Enter Divisor bits : ");
for (int i = 0; i < divisor_bits; i++)
    divisor[i] = Integer.parseInt(br.readLine());

tot_length = data_bits + divisor_bits - 1;
div = new int[tot_length];
rem = new int[tot_length];
crc = new int[tot_length];

for (int i = 0; i < data.length; i++)
    div[i] = data[i];

System.out.print("Dividend (after appending 0's): ");
for (int i = 0; i < div.length; i++)
```

```
    System.out.print(div[i]);
System.out.println();

for (int j = 0; j < div.length; j++)
    rem[j] = div[j];

rem = divide(div, divisor, rem);
for (int i = 0; i < div.length; i++)
    crc[i] = (div[i] ^ rem[i]);

System.out.println("\nCRC code:");
for (int i = 0; i < crc.length; i++)
    System.out.print(crc[i]);

System.out.println("\nEnter CRC code of " + tot_length + " bits:");
for (int i = 0; i < crc.length; i++)
    crc[i] = Integer.parseInt(br.readLine());

for (int j = 0; j < crc.length; j++)
    rem[j] = crc[j];

rem = divide(crc, divisor, rem);
for (int i = 0; i < rem.length; i++) {
    if (rem[i] != 0) {
        System.out.println("Error detected!");
        break;
    }
```

```
            if (i == rem.length - 1)
                System.out.println("No Error");
        }
        System.out.println("THANK YOU :)");
    }

    static int[] divide(int div[], int divisor[], int rem[]) {
        int cur = 0;
        while (true) {
            for (int i = 0; i < divisor.length; i++)
                rem[cur + i] = (rem[cur + i] ^ divisor[i]);
            while (rem[cur] == 0 && cur != rem.length - 1)
                cur++;
            if ((rem.length - cur) < divisor.length)
                break;
        }
        return rem;
    }
}
```

**OUTPUT:**

Enter number of data bits: 6

Enter data bits: 1 0 0 1 0 1

Enter number of bits in divisor: 4

Enter divisor bits: 1 1 0 1

Dividend (after appending 0's): 100101000

CRC code: 100101011

Enter CRC code of 9 bits:

100101011

No Error

THANK YOU :)




**RESULT:**

Thus, the Java program to simulate error correction using CRC was executed successfully.

## CONTENT BEYOND SYLLABUS

## IMPLEMENTATION OF STOP & WAIT PROTOCOL

**AIM:**

To implement the Stop and Wait Protocol using Java.

**PROCEDURE:**

**Sender:**

1. Initialize sequence number to 0.
2. Accept new packet and assign sequence number.
3. Send packet with sequence number.
4. Set timer for sent packet.
5. If acknowledgment is received correctly, move to next packet.
6. If timeout occurs, resend packet.
7. Stop transmission after all packets are sent.

**Receiver:**

1. Initialize NextFrameExpected = 0.
2. Receive frames continuously.
3. If frame is correct and expected, deliver to upper layer and increment expected frame.
4. Send acknowledgment.
5. Stop.

**PROGRAMS:**

**Sender.java**

```
import java.io.*;
import java.net.*;

public class Sender {
   Socket sender;
   ObjectOutputStream out;
   ObjectInputStream in;
   String packet, ack, str, msg;
   int n, i = 0, sequence = 0;

   Sender() {}

   public void run() {
      try {
         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
         System.out.println("Waiting for connection...");
         sender = new Socket("localhost", 2005);
         sequence = 0;
         out = new ObjectOutputStream(sender.getOutputStream());
         out.flush();
         in = new ObjectInputStream(sender.getInputStream());
         str = (String) in.readObject();
         System.out.println("Receiver > " + str);
         System.out.println("Enter data to send:");
         packet = br.readLine();
```

```
n = packet.length();


do {
   try {
      if (i < n) {
         msg = String.valueOf(sequence);
         msg = msg.concat(packet.substring(i, i + 1));
      } else if (i == n) {
         msg = "end";
         out.writeObject(msg);
         break;
      }
      out.writeObject(msg);
      sequence = (sequence == 0) ? 1 : 0;
      out.flush();
      System.out.println("Data sent > " + msg);
      ack = (String) in.readObject();
      System.out.println("Waiting for ACK...\n");
      if (ack.equals(String.valueOf(sequence))) {
         i++;
         System.out.println("Receiver > Packet received successfully.\n");
      } else {
         System.out.println("Timeout! Resending data...\n");
         sequence = (sequence == 0) ? 1 : 0;
      }
   } catch (Exception e) {}
} while (i < n + 1);
```

```
            System.out.println("All data sent. Exiting.");
        } catch (Exception e) {}
        finally {
            try {
                in.close();
                out.close();
                sender.close();
            } catch (Exception e) {}
        }
    }

    public static void main(String args[]) {
        Sender s = new Sender();
        s.run();
    }
}
```

**Receiver.java**

```
import java.io.*;
import java.net.*;

public class Receiver {
    ServerSocket receiver;
    Socket connection = null;
    ObjectOutputStream out;
    ObjectInputStream in;
    String packet, ack, data = "";
```

```
int i = 0, sequence = 0;

Receiver() {}

public void run() {
   try {
      receiver = new ServerSocket(2005, 10);
      System.out.println("Waiting for connection...");
      connection = receiver.accept();
      sequence = 0;
      System.out.println("Connection established.");
      out = new ObjectOutputStream(connection.getOutputStream());
      out.flush();
      in = new ObjectInputStream(connection.getInputStream());
      out.writeObject("Connected.");

      do {
         try {
            packet = (String) in.readObject();
            if (Integer.valueOf(packet.substring(0, 1)) == sequence) {
               data += packet.substring(1);
               sequence = (sequence == 0) ? 1 : 0;
               System.out.println("Receiver > " + packet);
            } else {
               System.out.println("Receiver > " + packet + " duplicate data");
            }
            if (i < 3) {
```

```
                out.writeObject(String.valueOf(sequence));
                i++;
            } else {
                out.writeObject(String.valueOf((sequence + 1) % 2));
                i = 0;
            }
        } catch (Exception e) {}
    } while (!packet.equals("end"));

    System.out.println("Data received = " + data);
    out.writeObject("Connection ended.");
  } catch (Exception e) {}
  finally {
    try {
      in.close();
      out.close();
      receiver.close();
    } catch (Exception e) {}
  }
}

public static void main(String args[]) {
  Receiver r = new Receiver();
  while (true) {
    r.run();
  }
}
```

}

**OUTPUT:**

Sender Side:

Waiting for connection...

Receiver > Connected.

Enter data to send: HELLO

Data sent > 0H

Receiver > Packet received

Data sent > 1E

Receiver > Packet received

...

All data sent. Exiting.

Receiver Side:

Waiting for connection...

Connection established.

Receiver > 0H

Receiver > 1E

Receiver > 0L

Receiver > 1L

Receiver > 0O

Data received = HELLO

**RESULT:**

Thus, the implementation of Stop & Wait Protocol was verified successfully.